

Junta de Andalucía
Consejería de Educación y Deporte

UD 1
Introducción a la programación
–
Módulo de Programación
1º DAW

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Autor: Fran Gómez

Objetivos



- Conocer los **conceptos básicos** relacionados con la programación y el diseño de aplicaciones
- Describir los **paradigmas** de programación más usados
- Clasificar los **lenguajes** de programación.
- Configurar el **entorno de desarrollo**
- Crear **algoritmos y programas** básicos

RA y CE implicados

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

a) Se han identificado los bloques que componen la estructura de un programa informático
b) Se han creado proyectos de desarrollo de aplicaciones
c) Se han utilizado entornos integrados de desarrollo.
d) Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.
e) Se ha modificado el código de un programa para crear y utilizar variables.
f) Se han creado y utilizado constantes y literales.
g) Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.
h) Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.
i) Se han introducido comentarios en el código.

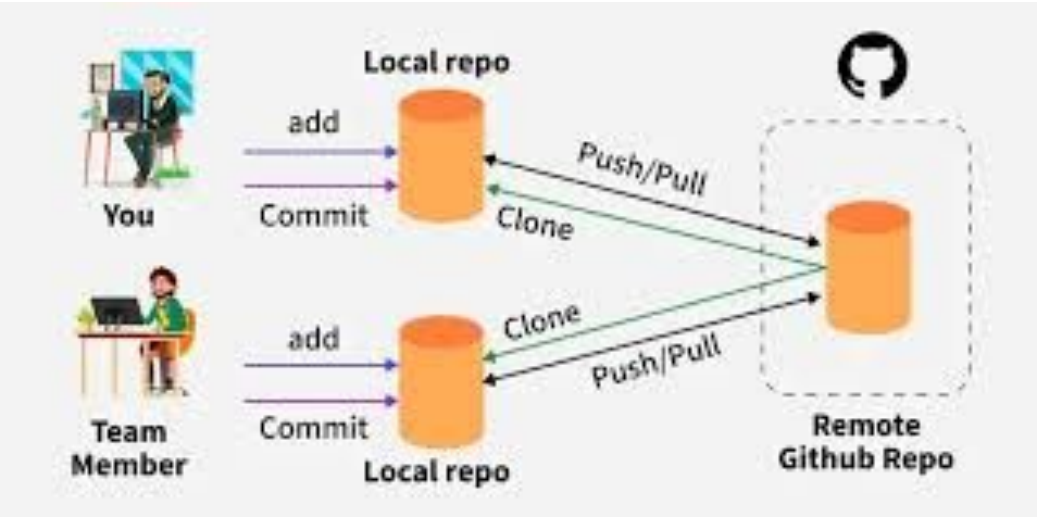
Índice de contenidos

1. Introducción
   1.1. Evolución de la programación
   1.2. Conceptos básicos
   1.3. Clasificación de los lenguajes
2. Paradigmas de programación
   2.1. Declarativa
   2.2. Imperativa
   2.3. Estructurada
   2.4. Modular
3. Pseudocódigo
4. Diagramas de flujo
5. Codificar en un lenguaje de programación:
   5.1. Empezando con Python
   5.2. Empezando con Java
6. Sistema Control de Versiones: Git
7. Estructura de un programa
8. Elementos de un programa
   8.1. Palabras reservadas
   8.2. Operadores
   8.3. Tipos de datos
   8.4. Conversiones
9. Ejercicios
10. Proyecto
11. Examen
12. Recursos y referencias

Empezando con Git



1. Instalar Git
2. Configurar Git
3. Crear cuenta en GitHub
4. Crear repositorio en GitHub
5. Crear nuestro programa, iniciar repositorio local, y subirlo



Proyecto local
▼
git init
▼
git add
▼
git commit
▼
git push
▼
GitHub

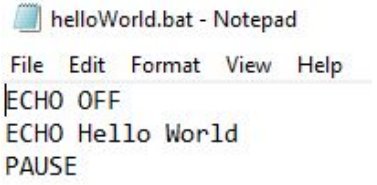
Introducción

Mi primer programa en bash/shell







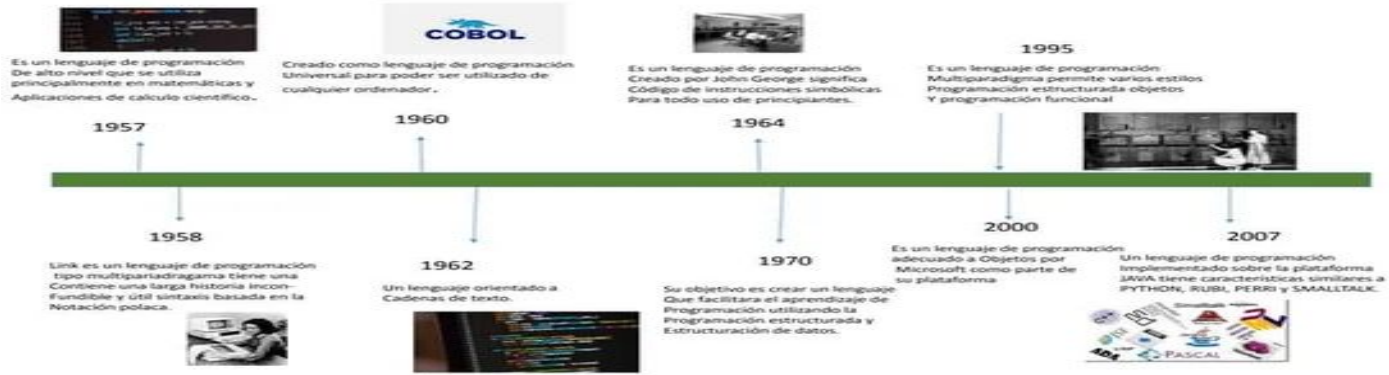


Evolución de la programación

Ejercicio 1

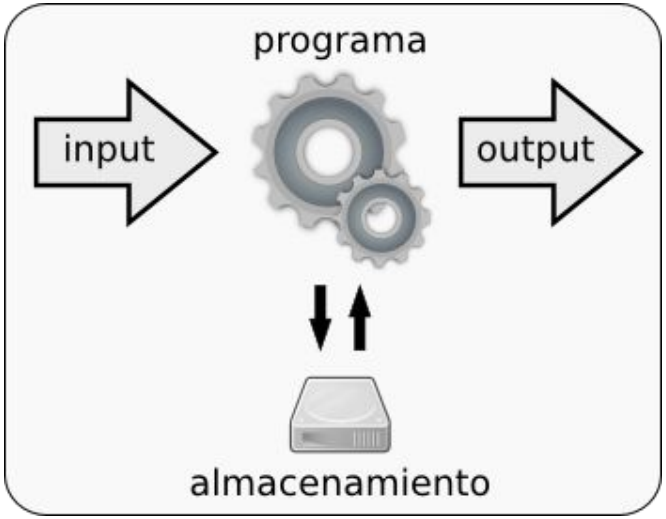
Crea una línea del tiempo con los items que más destacarías en la historia de la programación.

1. Año
2. Nombre (logotipo)
3. Creador
4. Alguna característica



Conceptos básicos

- Programación
- Algoritmo
- Programar
- Código
- Lenguaje binario
- Lenguaje ensamblador
- Lenguajes de alto nivel



- Entrada
- Salida
- Cambio de estado
- Lógica
- Datos

Conceptos básicos

Ejercicio 2

Describe en qué consiste la programación y qué diferencia hay entre los algoritmos y Código.

Clasificación de los lenguajes

- Propósito:
  - General vs específico
- Tipo:
  - Scripting (o interpretado)
  - Compilado
  - Marcado
- Uso:
  - Desarrollo Web
  - Juegos
  - Administración de Sistemas
  - Móvil / Multiplataforma
  - IA, BigData, Machine Learning…
- Plataforma:
  - Web
    - Frontend
    - Backend
  - Desktop
  - Móvil
  - Embedded
- Paradigma:
  - Imperativo
  - Estructurado
  - Orientado a objetos
  - Declarativo

Clasificación de los lenguajes

Ejercicio 3

Crea una tabla comparativa de los lenguajes más populares, clasificándolo
parámetros ant

¿Cuál elegiría
programar? ¿po

Lenguaje	Propósito	Tipo	Uso Principal	Plataforma	Ventajas	Desventajas
Python	General	Interpretado	Ciencia de datos, ML, desarrollo web	Web, ciencia de datos, ML	Fácil de aprender, gran comunidad	Velocidad de ejecución
Java						
JavaScript						
PHP						
C#						
C						
C++						
GO						
Rust						
Swift						
Ruby						
Bash/Batch/PowerShell						

Paradigmas de programación

- **Imperativa** ⇒ Instrucciones paso a paso. “Cómo”
  - **Estructurada** o procedural => Instrucciones siguiendo estructuras. Eliminar Goto.
  - **Modular** ⇒ Divide el programa en partes. Subrutinas.
- **Declarativa** ⇒ Resultado deseado. “Qué”
  - **Funcional** ⇒ funciones matemáticas
  - **Lógico** ⇒ reglas lógicas
- **Orientada a objetos** ⇒ Agrupa Datos y Lógica

Programación Declarativa

Queremos programar la obtención de los clientes de Madrid mayores de 30 años.

```sql
SELECT nombre, apellido
FROM clientes
WHERE ciudad = 'Madrid' AND edad > 30;
```

Declaración, no instrucción: No le dices al ordenador cómo buscar a los clientes, simplemente **declaras** que quieres los nombres y apellidos de aquellos que cumplen las condiciones.

Foco en el resultado: Te centras en el **qué** quieres obtener, no en el **cómo** lo vas a obtener.

Motor de base de datos: El motor de base de datos se encarga de optimizar la consulta y encontrar los datos que cumplen con los criterios especificados.

Queremos programar la elaboración de un sándwich

```c
#include <stdio.h>

int main() {
    printf("Tomando una rebanada de pan\n");
    printf("Untando mantequilla\n");
    printf("Colocando queso\n");
    printf("Tomando otra rebanada\n");
    printf("Cerrando el sándwich\n");
    return 0;
}
```

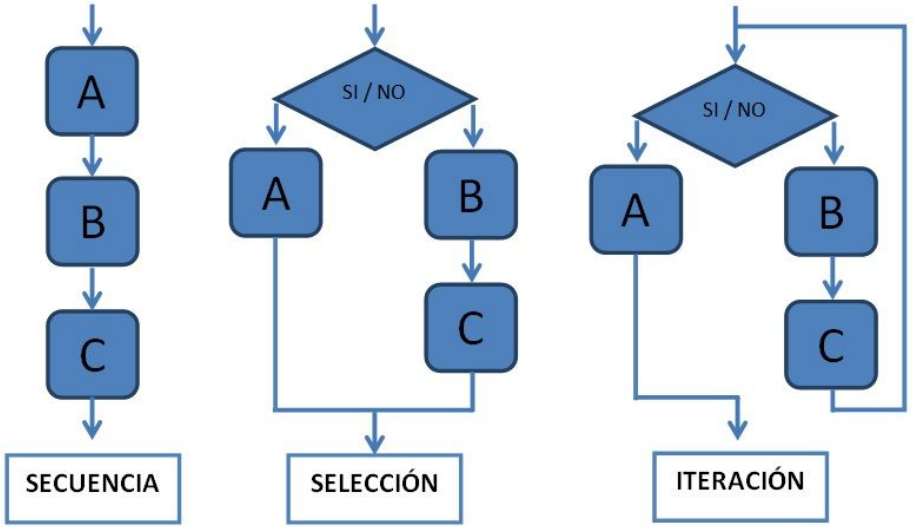
Instrucciones secuenciales: Cada línea de código representa una instrucción específica que se ejecuta una después de la otra.

Estado mutable: El estado del sándwich va cambiando a medida que se ejecutan las instrucciones.

Foco en el proceso: Nos enfocamos en describir *cómo* hacer el sándwich, paso a otro.

Programación Estructurada

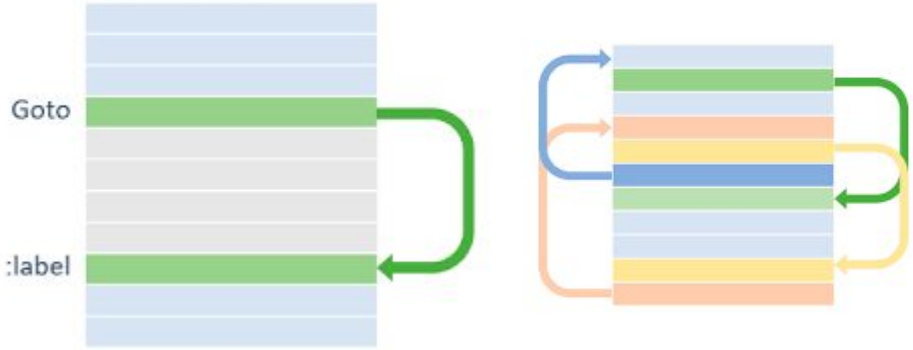
- Secuencial
- Alternativa
- Iterativa



Ideas capacitación
Programación estructurada.

Programación Estructurada

Sentencia GOTO




```
goto etiqueta;

...
...
...

etiqueta: sentencia;
```

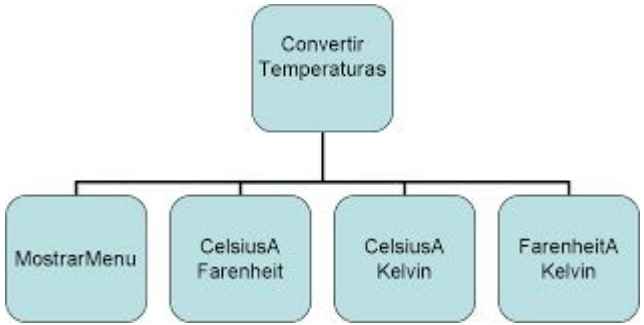
Código espagueti



Java Hispano

Programación Modular

- Los programas crecen y se hacen más complejos ⇒ dividirlos en módulos
- Módulo = subprograma o rutina ⇒ funciones
- Librerías





Clasificación de los lenguajes

Ejercicio 3 (Continuación)

Completa el ejercicio 3 añadiendo una columna para indicar el paradi

¿Cambi
¿por q

Lenguaje	Propósito	Tipo	Uso Principal	Plataforma	Ventajas	Desventajas	Paradigma
Python	General	Interpretado	Ciencia de datos, ML, desarrollo web	Web, ciencia de datos, ML	Fácil de aprender, gran comunidad	Velocidad de ejecución	Multiparadigma
Java							
JavaScript							
PHP							
C#							
C							
C++							
GO							
Rust							
Swift							
Ruby							
Bash/Batch/PowerShell							

Your Wooclap question will appear here

1. Install the **Chrome** or **Firefox** extension 🧩
2. Click on “Add a Wooclap vote” and make sure you are logged into your Wooclap account 👨‍💻
3. Ensure you are in **presentation mode** on Google Slides 🖥️

wooclap

Estructura

ENCABEZADOS / I

PROGRAMA nomb

CONSTANTES

Declaración y a

VARIABLES

Declaración de

FUNCIONES / MÓD

INICIO

Instrucciones d

FIN

```java
// Importaciones
import java.util.Scanner;

public class ProgramaComple

    // Declaración de varia
    static int edad = 25;
    static String nombre =

    // Funciones / Métodos
    public static String sa
        // Devuelve un mens
        return "Hola " + pe
    }

    public static double ca
        // Calcula el área
        return Math.PI * ra
    }

    // Bloque principal / E
    public static void mair
        String mensaje = sa
        System.out.println(

        double radio = 5;
        double area = calcularAreaCirculo(radio);
        System.out.printf("El área del círculo de radio %.2f es %.2f\n", radio, area)}
}
```

```python
# Importaciones
import math

# Declaración de variables
edad = 25
nombre = "Ana"

# Funciones / Módulos
def saludar(persona, edad):
    """Devuelve un mensaje de saludo con nombre y edad"""
    return f"Hola {persona}, tienes {edad} años."

def calcular_area_circulo(radio):
    """Calcula el área de un círculo"""
    return math.pi * radio ** 2

# Bloque principal / Ejecución
if __name__ == "__main__":
    mensaje = saludar(nombre, edad)
    print(mensaje)

    radio = 5
    area = calcular_area_circulo(radio)
    print(f"El área del círculo de radio {radio} es
{area:.2f}")
```

Elementos de un programa

1. **Entrada**
2. Procesamiento
3. Salida
4. Palabras reservadas
5. Comentarios

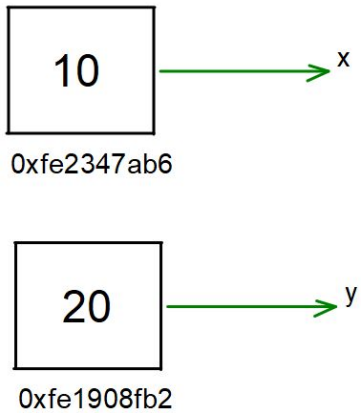


- **Datos**: La información que el programa recibe del usuario o de otros dispositivos para realizar sus cálculos o tomar decisiones.
- **Dispositivos** de **entrada**: Teclado, mouse, escáner, micrófonos, etc.

Elementos de un programa

1. Entrada
2. **Procesamiento**
3. Salida
4. Palabras reservadas
5. Comentarios

```
instrucción 1
instrucción 2
      .
      .
      .
instrucción n
```



- **Instrucciones:** Órdenes o sentencias. Unidad mínima.
- **Variables**: Espacios en la memoria del ordenador donde se almacenan los datos que el programa está utilizando.
- **Constantes**: como una variable cuyo valor inicial no cambia a lo largo del programa
- **Literales**: datos “*a pelo*” en el código.

Elementos de un programa

1. Entrada
2. **Procesamiento**
3. Salida
4. Palabras reservadas
5. Comentarios



- **Operadores**: Símbolos que permiten realizar operaciones matemáticas, lógicas o de comparación sobre los datos.

  +, -, *, /, =

- **Estructuras de control**: Mecanismos que permiten controlar el flujo de ejecución del programa, como las condicionales (si, entonces, sino) y los bucles (para, mientras).

Elementos de un programa

1. Entrada
2. **Procesamiento**
3. Salida
4. Palabras reservadas
5. Comentarios

- **Expresiones:** Son combinaciones de literales, constantes, variables y operadores para ejecutar una operación.

  7 + 3 edad < 12 PI * r^2

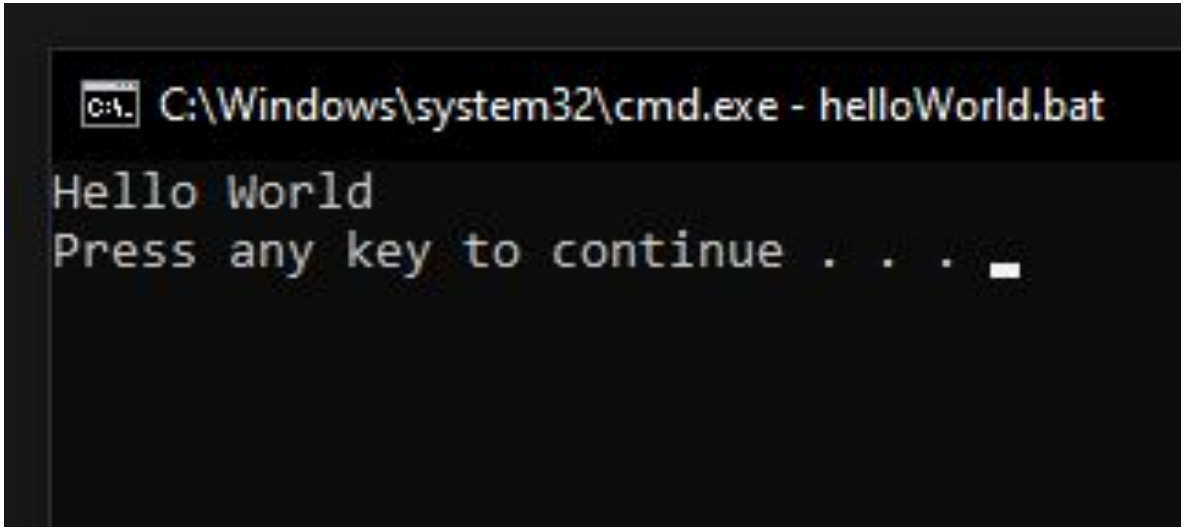
- **Asignación**: Operación que toma el valor de una expresión y lo almacena en una variable

  nombre = “Fran” suma = 2 + 3

Elementos de un programa

1. Entrada
2. Procesamiento
3. **Salida**
4. Palabras reservadas
5. Comentarios

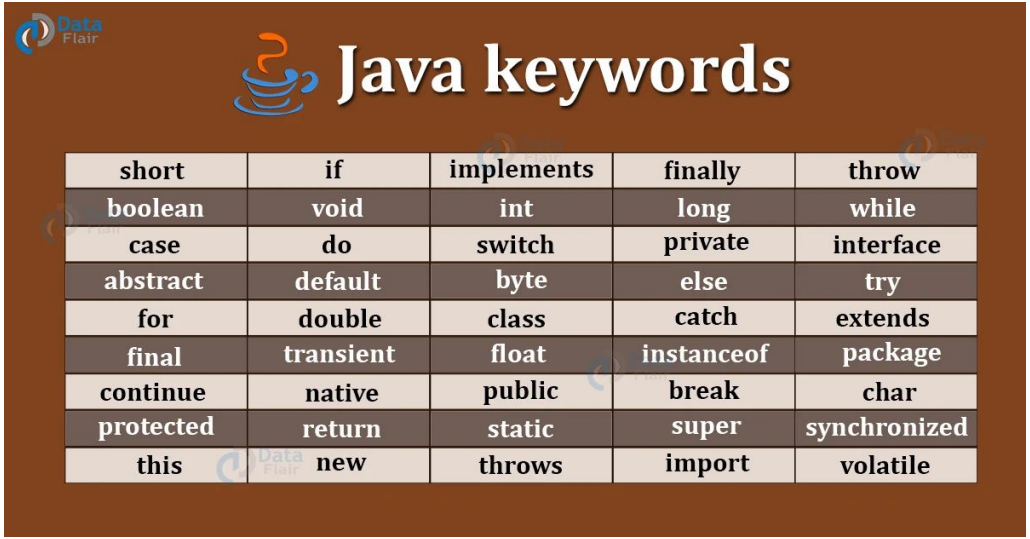
- **Resultados**: La información que el programa genera a partir del procesamiento de los datos de entrada.
- **Dispositivos** de salida: Pantalla, impresora, altavoces, etc.



Elementos de un programa

1. Entrada
2. Procesamiento
3. Salida
4. **Palabras reservadas**
5. Comentarios

Comandos que entiende el programa



No podemos usarlas para nombres de variables, algoritmos, etc.

Elementos de un programa

1. Entrada
2. Procesamiento
3. Salida
4. Palabras reservadas
5. **Comentarios**

- Texto que añade claridad al código, explicando qué hace cada parte del programa. Son muy útiles para documentar el código y facilitar su comprensión por parte de otros programadores o por uno mismo en el futuro.

```
# Imprime el resultado
print (resultado);
```

Ejemplo de Programa

Programa que calcula el área de un círculo.

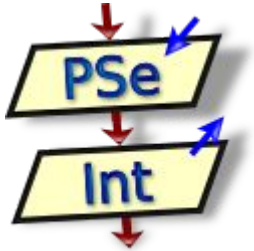
Entrada: El radio del círculo (un número ingresado por el usuario).

Procesamiento: Una variable llamada "radio" para almacenar el valor ingresado. Una constante llamada "pi" con el valor 3.14159. Una sentencia que multiplica el radio por sí mismo y luego por pi para calcular el área.

Salida: El resultado del cálculo (el área del círculo), mostrado en pantalla.

Comentarios: Explicaciones sobre cómo se realiza el cálculo y el significado de cada variable.

Pseudocódigo



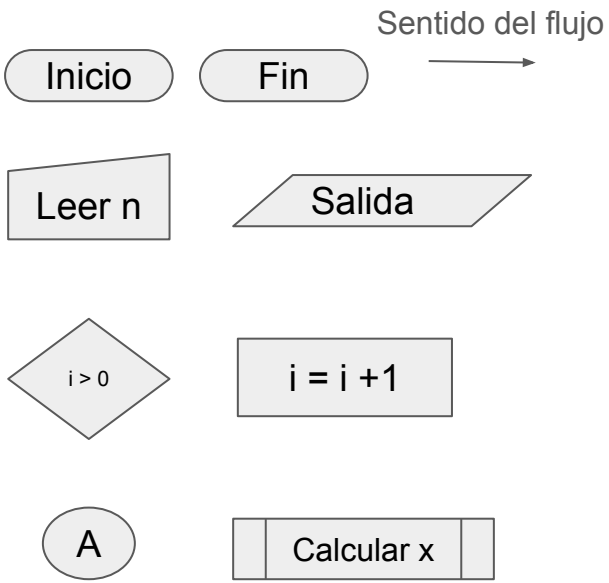

```
1  Algoritmo calcular_area_circulo
2      // Declaración de variables
3      Definir radio, area Como Real
4
5      // Entrada de datos
6      Escribir "Ingrese el radio del círculo: "
7      Leer radio
8
9      // Cálculo del área
10     area ← pi * radio * radio
11
12     // Salida de resultados
13     Escribir "El área del círculo es: ", area
14
15 FinAlgoritmo
```

Crear un algoritmo sin usar un lenguaje de programación concreto.

- Lenguaje informal → Cada uno con sus palabras
- Ayuda a ver la lógica antes de pasar a codificar en un lenguaje real
- Facilita comunicación entre programadores
- Sirve para documentar el código
- Útil para aprender a programar

Diagramas de flujo

Representación gráfica de un programa



ANSI
American National Standards Institute



Ejercicio

Ejercicio 4

Escribe un algoritmo y dibuja el diagrama para calcular el área de un círculo.

Debe solicitar al usuario los datos de entrada y mostrar el resultado cuando finalice.

No es necesario que uses una sintaxis de un lenguaje concreto sino tus propias palabras.

Ejemplos de Algoritmos

RECETA DE COCINA: Cómo preparar una gelatina
1. Inicio
2. Hervir en una olla un litro de agua
3. Agregar el sobre de gelatina
4. Dejarlo a fuego lento cinco minutos revolviendo constantemente
5. Dejar reposar para que tome temperatura ambiente
6. Refrigerar dos hora hasta que cuaje
7. Fin

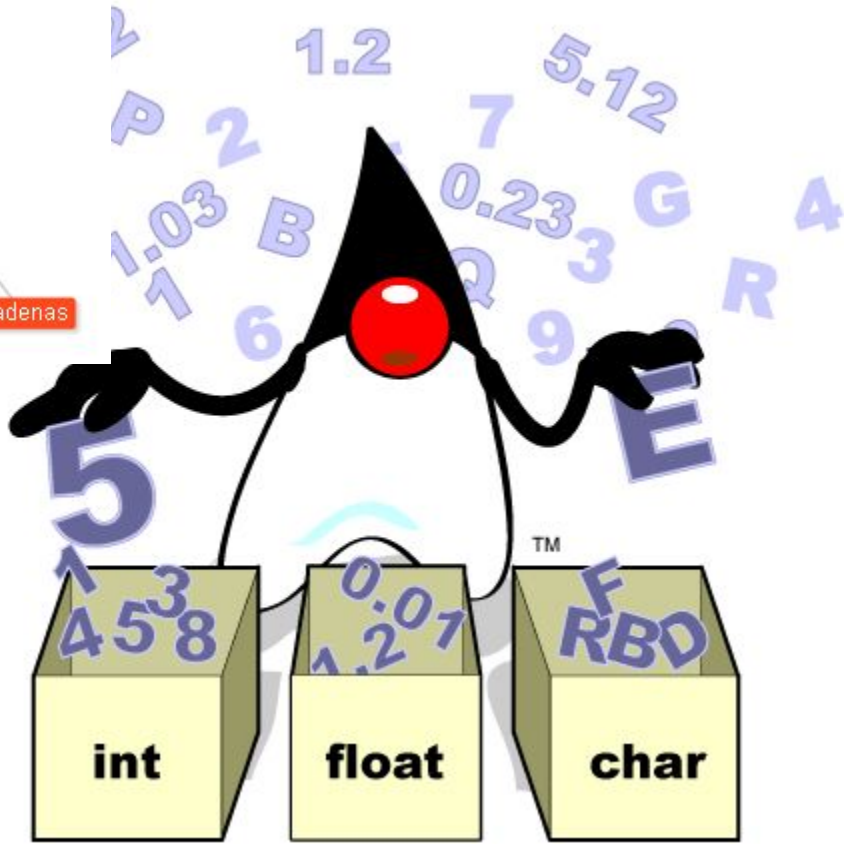


Tipos de datos



Operadores Lógicos

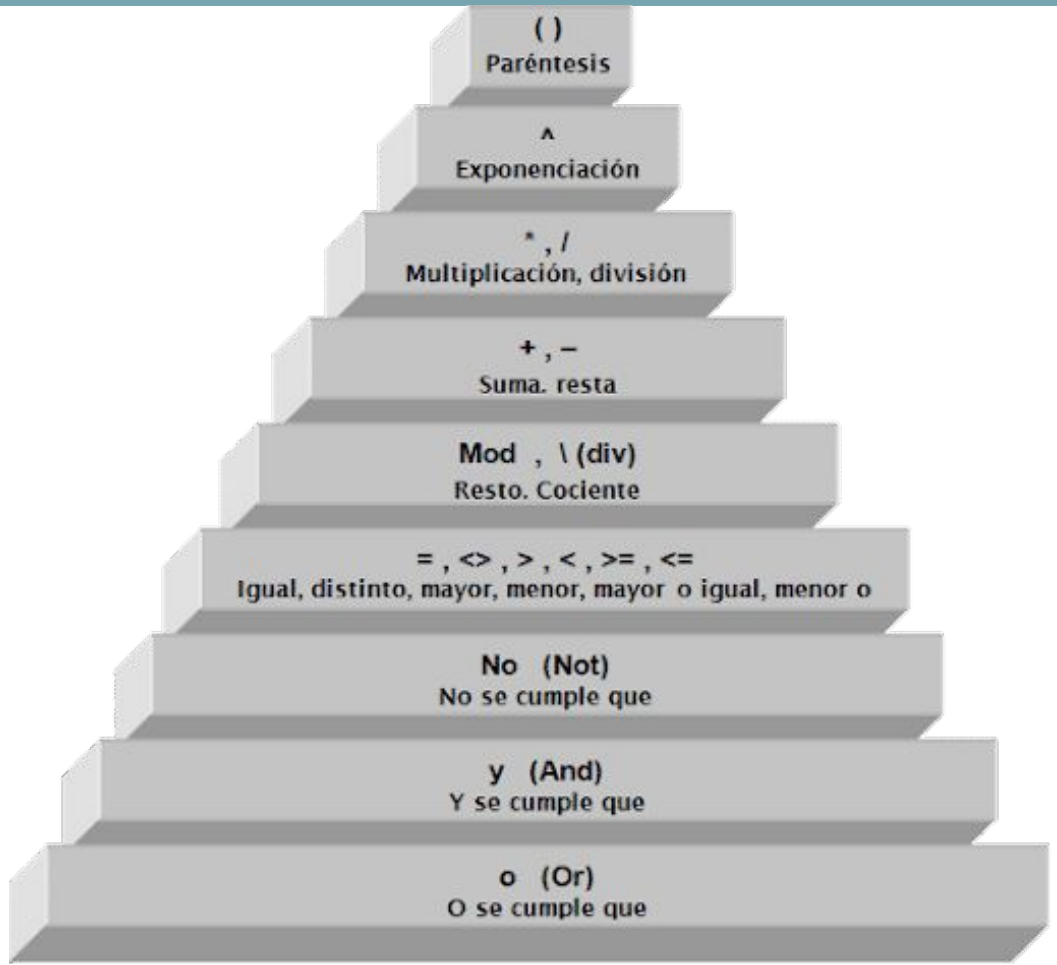
P	Q	~P	~Q	P o Q	P y Q
Verdadero	Verdadero	Falso	Falso	Verdadero	Verdadero
Verdadero	Falso	Falso	Verdadero	Verdadero	Falso
Falso	Verdadero	Verdadero	Falso	Verdadero	Falso
Falso	Falso	Verdadero	Verdadero	Falso	Falso



Tipos de operadores

Tipo de operador	Símbolos python	Descripción
Aritméticos	`+`, `-`, `*`, `/`, `%`, `**`, `//`	Operaciones matemáticas: suma, resta, multiplicación, división, módulo, potencia, división entera
Relacionales / Comparación	`==`, `!=`, `>`, `<`, `>=`, `<=`	Comparan valores y devuelven `True` o `False`
Lógicos	`and`, `or`, `not`	Combina expresiones booleanas
De asignación	`=`, `+=`, `-=`, `*=`, `/=`, `%=`	Asignan valores a variables, con operaciones combinadas
Concatenación	`+`	Pega dos cadenas de caracteres
De identidad	`is`, `is not`	Comprueba si dos objetos son **el mismo objeto** en memoria
De pertenencia	`in`, `not in`	Comprueba si un elemento **pertenece a una colección**
Bit a bit / a nivel de bits	`&`, `` ` ``	`, ^, ~, <<, >>` `

Precedencia de los Operadores

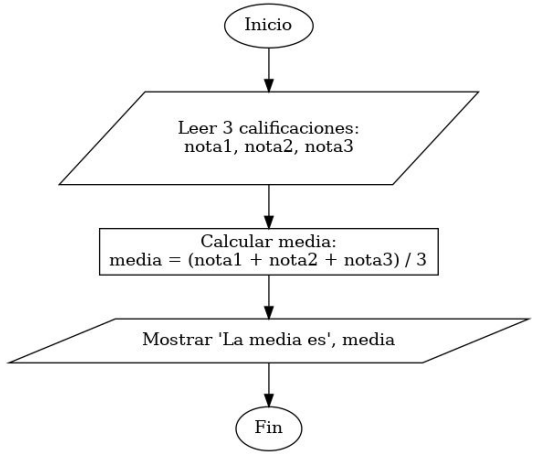


Ejercicio

Ejercicio 5

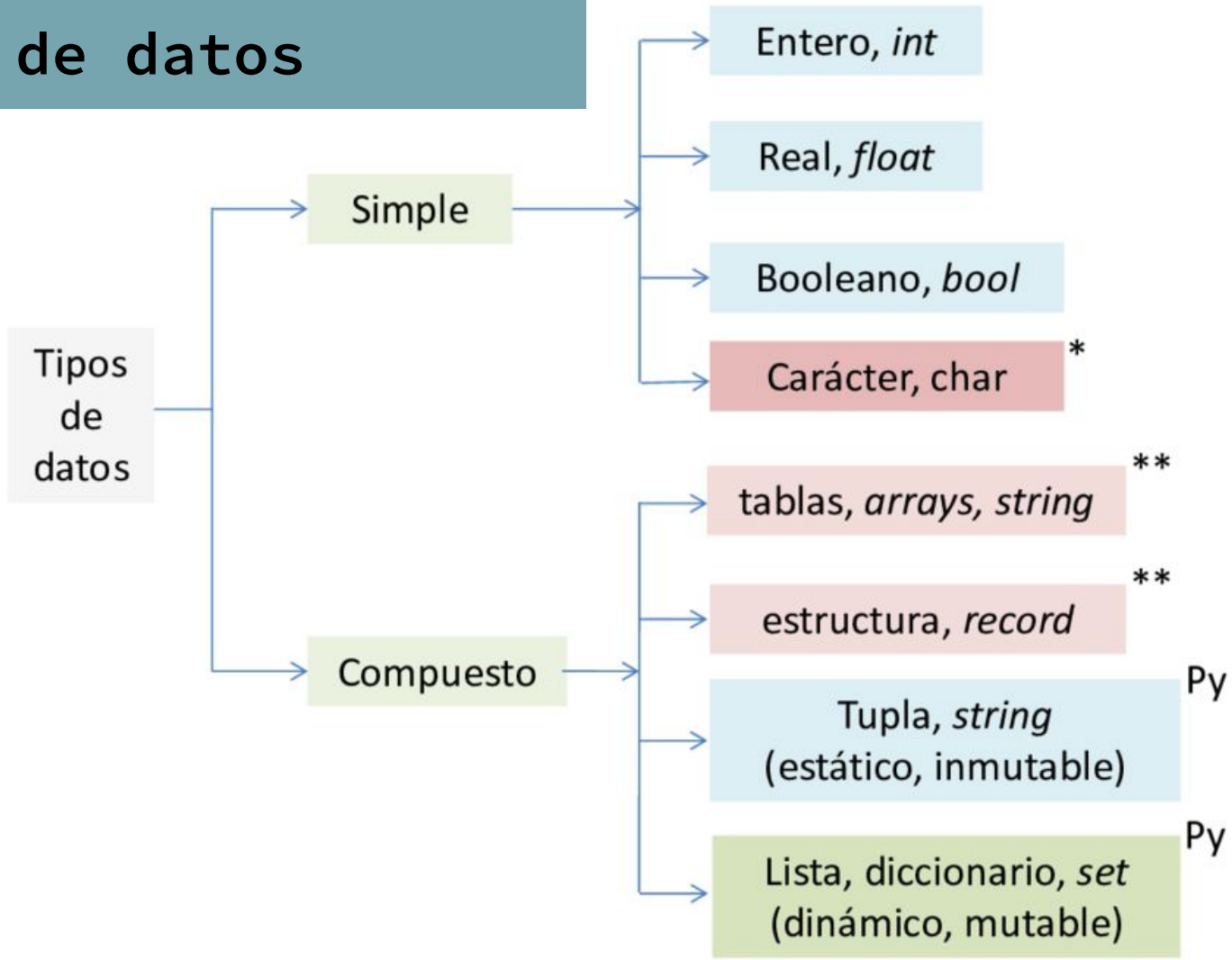
Diagrama y algoritmo que calcule la nota media de tres actividades.

Usa los paréntesis para modificar la precedencia y así realizar correctamente el cálculo



1. Inicio
2. Leer valores de: Calificación1, Calificación2, Calificación3
3. Promedio = (Calificación1 + Calificación2 + Calificación3)/3

Tipos de datos



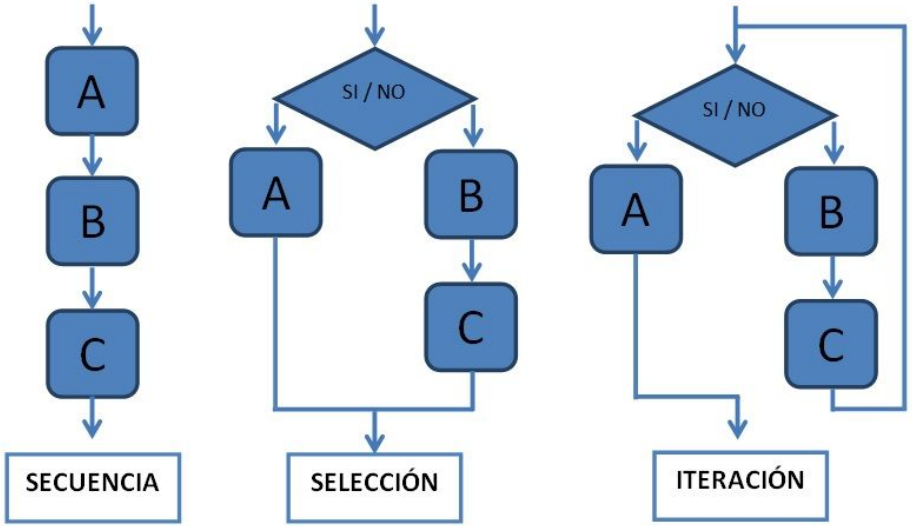
Ejercicio

Ejercicio 5 (continuación)

Identifica y añade el tipo dato de cada variable.

Estructuras

- Secuencial
- Selectiva o Alternativa
- Iterativa o Repetitiva



Ejercicio

Ejercicio 6

Escribe un algoritmo que a partir del año de nacimiento introducido muestre como resultado si esa persona es mayor de edad o no.

1. Inicio
2. Leer Fecha de nacimiento, Fecha Actual
3. Edad = Fecha actual – Fecha de nacimiento
4. Edad >=18
5. SI
6. "Credencial Autorizada"
7. No
8. "Credencial No Autorizada"
9. FIN

Ejercicio

Ejercicio 7

Escribe un algoritmo que a partir de tres calificaciones, calcule la media ponderada (nota1 20%, nota2 50% y nota3 30%), y a partir de la media, indique si el alumno está aprobado o no.

Matemático:
Calcular el promedio de tres calificaciones; si es mayor a 6, mostrar aprobado, si no mostrar reprobado.

1. Inicio
2. Leer valores de: Calificación1, Calificación2, Calificación3
3. Promedio = (Calificación1 + Calificación2 + Calificación3)/3
4. **Si promedio** >5.9, entonces
   Escribir "Aprobado"
   Si no
   Escribir "Reprobado"
   Fin Si
5. FIN

¿Qué es Python?

Alto Nivel
Sintaxis clara y cercana al lenguaje humano.

Interpretado
Ejecución directa instrucción por instrucción.

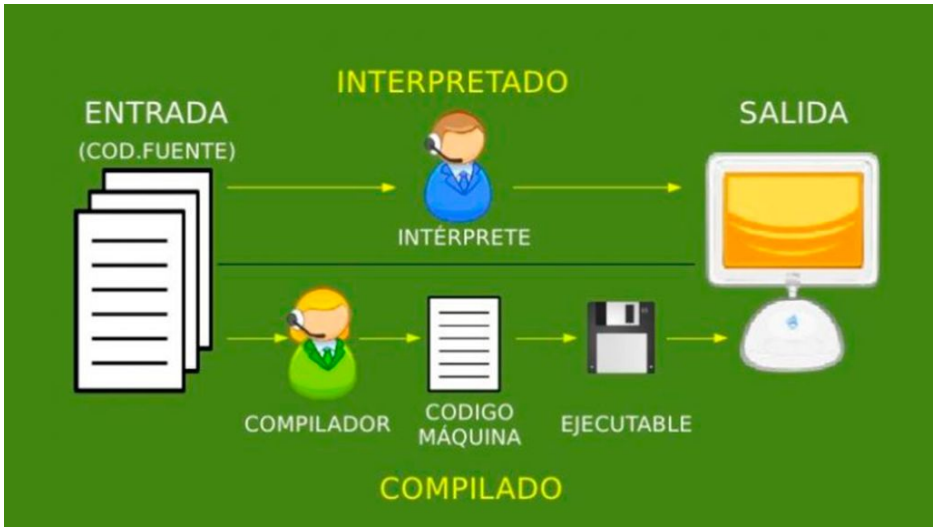
Tipado Dinámico
Detección de tipos en tiempo de ejecución.

Multiparadigma
Soporta POO, funcional y estructurado.

Python: funcionamiento

Código Fuente

Script `.py` escrito por el desarrollador.



Python: Intérprete y Entorno



IDLE
Integrated Development and Learning Environment

Preparación del entorno:

1. Instalamos **Python** descargando el instalador de su web: python.org
2. Instalamos el IDE **IDLE** (se incluye con la instalación de Python)
3. Probamos el intérprete de comando a través del IDE.
4. Creamos un fichero de código fuente, introducimos el código para crear un programa *Hola mundo* y lo ejecutamos:

```
print("Hola mundo!")
```

Python: Palabras reservadas del lenguaje

Regla Fundamental

Identificadores protegidos por el intérprete. Prohibido usarlos como nombres de variables o funciones.

⚠️ **Sensibles a mayúsculas: `True`, `False` y `None` llevan mayúscula inicial.**

MUESTRA DE KEYWORDS

False	True	None
if	else	for
def	return	import
and	or	not

Python: Tipos de Datos Básicos (Data Types)

Enteros (`int`): Números sin decimales (ej. `10`, `-5`, `0`).

Decimales / Coma flotante (`float`): Números con punto decimal (ej. `3.14`, `-0.5`).

Cadenas de texto (`str`): Texto delimitado por comillas simples o dobles (ej. `"Hola"`, `'Python'`).

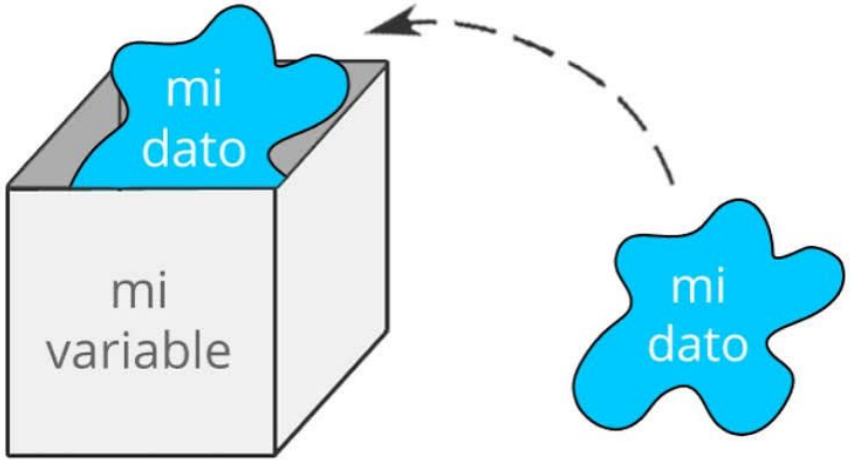
Booleanos (`bool`): Estados lógicos, solo pueden ser `True` (Verdadero) o `False` (Falso).

Python

```python
type(25)     # <class 'int'>
type("Hola") # <class 'str'>
```

Python: Variables (Contenedores de datos)

Una variable es un nombre que se le asigna a un espacio en memoria para almacenar un valor y reutilizarlo después.



Reglas básicas de nombrado:

1. Deben ser descriptivas (`edad` en lugar de `x`).
2. No pueden empezar con números ni contener espacios.
3. Usar la convención *snake_case* (palabras en minúscula separadas por guion bajo, ej. `precio_total`).

```python
# Creación de variables
nombre = "Ana"          # str
edad = 20               # int
estatuta = 1.65         # float
es_estudiante = True # bool
```

En Python las variables:

- Se declarar e inicializan a la vez (no se pueden declarar previamente)
- El tipado es automático

Operador de asignación (`=`). No confundir con comparación.

Python: Operadores

Herramientas que permiten realizar acciones y cálculos sobre variables y valores.

A. Operadores Aritméticos

Operador	Descripción	Ejemplo	Resultado
`+`	Suma	`5 + 2`	`7`
`-`	Resta	`5 - 2`	`3`
`*`	Multiplicación	`5 * 2`	`10`
`/`	División flotante	`5 / 2`	`2.5`
`//`	División entera (trunca decimales)	`5 // 2`	`2`
`%`	Módulo (resto de la división)	`5 % 2`	`1`
`**`	Potencia	`5 ** 2`	`25`

Python: Operadores

Herramientas que permiten realizar acciones y cálculos sobre variables y valores.

B. Operadores de Comparación (Devuelven un `bool`)

- `==` Igual a (`5 == 5` es `True`)
- `!=` Diferente de (`5 != 2` es `True`)
- `>` / `<` Mayor que / Menor que
- `>=` / `<=` Mayor o igual / Menor o igual

C. Operadores Lógicos

- `and` : Devuelve `True` si **ambas** condiciones son verdaderas.
- `or` : Devuelve `True` si **al menos una** condición es verdadera.
- `not` : Invierte el valor booleano (`not True` pasa a ser `False`).

Python: Conversión de tipos (Casting)

Transformación de un dato de un tipo a otro.

Conversión Implícita (Automática)

Python convierte automáticamente ciertos tipos cuando realiza operaciones mixtas para evitar pérdida de datos:

```
num_int = 10
num_float = 2.5
resultado = num_int + num_float  # resultado es 12.5 (float)
```

Conversión Explícita (Manual)

Uso de las funciones integradas `int()`, `float()`, `str()` y `bool()`:

```
# Convertir un texto a número entero
texto_edad = "25"
numero_edad = int(texto_edad)  # 25 (int)

# Convertir un número a texto para concatenar
puntos = 100
mensaje = "Puntaje total: " + str(puntos)  # "Puntaje total: 100"
```

Python: Conversión de tipos

La función `input()` siempre devuelve una cadena de texto (`str`).
Para hacer cálculos con lo que escribe el usuario, es obligatorio realizar la conversión explícita:

Python

```python
# Esto fallará al intentar sumar si no se convierte:
edad_ingresada = input("Ingresa tu edad: ") # Retorna p. ej. "20"
edad_numero = int(edad_ingresada)           # Convertimos a int
edad_futura = edad_numero + 1               # Operación válida: 21
```

Python: Built-in functions

Python como cualquier lenguaje incluye algunas herramientas (lo que llamábamos subrutinas, módulos o funciones) para que las usemos:

Herramienta	¿Qué hace? (Acción)	¿Qué entra? ()	¿Qué sale / devuelve?
`print(...)`	Muestra un mensaje en pantalla	Cualquier dato o texto	Nada (solo imprime)
`input(...)`	Detiene el programa y pide texto al usuario	El mensaje de pregunta	**Siempre un texto (`str`)**
`type(...)`	Revela el tipo de dato	Cualquier variable o valor	El tipo (`int` , `float` , `str` ...)
`int(...)` / `float(...)` / `str(...)`	Transformadores (conversión de tipo)	El valor a transformar	El valor en el nuevo tipo

Python: Built-in functions

Si una herramienta devuelve un valor (como `input()` o `int()`), debemos guardarlo en una **variable** usando el signo `=` si queremos reutilizarlo más adelante.

Ejemplo:

```
nombre = input("Introduzca nombre")
print(nombre)
```

¿Qué muestra el ejemplo anterior por pantalla?

Python: Comentarios

Un **comentario** es un texto en el código fuente que el intérprete o compilador ignora por completo durante la ejecución. Su único propósito es servir de lectura para los seres humanos (desarrolladores, compañeros de equipo o el propio autor en el futuro).

1. ¿Por qué son importantes en programación?

- **Documentación e intención:** Explican el *porqué* se tomó una decisión lógica compleja, no solo el *qué* hace el código.
- **Trabajo en equipo:** Ayudan a que otros programadores entiendan rápidamente la estructura del proyecto.
- **Pruebas y depuración (*debugging*):** Permiten "desactivar" temporalmente líneas de código sin borrarlas para aislar errores.

2. Sintaxis de Comentarios en Python

A diferencia de lenguajes como C++, Java o JavaScript (que usan `//` y `/* ... */`), Python utiliza el símbolo almohadilla/numeral (#) y cadenas de texto multilínea.

Cadenas multilínea con triples comillas (`"""` o `'''`): Si una cadena de texto no se a
a una función, el intérprete la lee pero no ejecuta nada con ella, usándose en la práctic

```
"""
Este es un comentario que ocupa
múltiples líneas utilizando triples comillas.
Es ideal para explicaciones más largas.
"""
print("Hola mundo")
```

Python: Comentarios

Buenas Prácticas

1. Evitar comentarios redundantes u obvios

El código debe ser lo más autoexplicativo posible mediante el uso de buenos nombres de variables.

```
# ❌ MAL: Comentario innecesario
x = 10  # Asigna el valor 10 a la variable x

# 🟢 BIEN: El código habla por sí mismo, no requiere comentario
limite_intentos = 10
```

2. Explicar el *POR QUÉ*, no el *QUÉ*

```
# ❌ MAL: Explica lo que ya se ve en el código
precio = precio * 1.05  # Multiplica el precio por 1.05

# 🟢 BIEN: Explica el motivo de negocio detrás de la operación
precio = precio * 1.05  # Ajuste automático del 5% por tasa de inflación anual
```

3. Mantener los comentarios actualizados y el código autoexplicativo

Python: Ejemplo de resumen


```python
# 1. Entrada de datos y Conversión Explícita (str -> float / int)
precio_unitario = float(input("Ingrese el precio del producto: "))
cantidad = int(input("Ingrese la cantidad deseada: "))

# 2. Uso de Variables y Operadores Aritméticos
subtotal = precio_unitario * cantidad

# 3. Operadores de Comparación y Lógicos (Genera un bool)
aplica_descuento = subtotal > 100 and cantidad >= 3

# 4. Mostrar resultados convirtiendo tipos a str para concatenar
print("Monto subtotal: $" + str(subtotal))
print("¿Elegible para descuento especial?: " + str(aplica_descuento))
```

Python: Ejercicios

Para los siguientes ejercicios vamos a realizar los 3 pasos:

- ➔ Diagrama de flujo (opcional)
- ➔ Pseudocódigo (opcional)
- ➔ Código en Python (En blog todo en la misma entrada y en Github)

Python: Ejercicios

8. Hola mundo con variables:

Escribe un programa que muestre por pantalla “Hola, *nombre*” donde *nombre* es una variable que debes declarar e inicializar previamente con el valor de tu nombre.

Ampliación: Modifica el código para que la variable nombre sea introducida desde la consola por teclado.

9. Ficha del Programador

Objetivo: Practicar `input()`, `type()` y conversión explícita de datos.

Escribe un programa que solicite al usuario:

1. Su nombre.
2. Su año de nacimiento.
3. Su altura en metros (ejemplo: `1.75`).

Ejemplo de salida esperada

```
--- FICHA REGISTRADA ---
Nombre: Carlos (Tipo: <class 'str'>)
Edad: 24 años (Tipo: <class 'int'>)
Altura: 1.75 m (Tipo: <class 'float'>)
```

El programa debe:

- Convertir el año de nacimiento a número entero y calcular su edad aproximada al año actual (2026).
- Convertir la altura a decimal (`float`).
- Imprimir en pantalla los tipos de datos de cada variable convertida usando la función `type()`.
- Mostrar un mensaje final con toda la información integrada.

Python: Ejercicios

Ejercicio 10: Conversor de Temperatura

Objetivo: Aplicar la fórmula de conversión de Celsius a Fahrenheit usando operadores aritméticos y precedencia.

- **Enunciado:**
  La fórmula para convertir grados Celsius (C) a Fahrenheit (F) es:

$$F = (C \times \frac{9}{5}) + 32$$

  Escribe un script en Python que pida al usuario la temperatura en grados Celsius, realice el cálculo y muestre la equivalencia en Fahrenheit.

- **Ejemplo de Salida Esperada:**

```
Ingrese la temperatura en ºC: 25.5
25.5 ºC equivalen a 77.9 ºF
```

Python: Ejercicios

Ejercicio 11: Reparto de Caramelos (División Entera y Módulo)

Objetivo: Diferenciar el uso de la división flotante `/`, la división entera `//` y el residuo `%`.

- **Enunciado:** Un profesor tiene un paquete de caramelos para repartir en partes iguales entre sus estudiantes. Escribe un programa que solicite:
  - La cantidad total de caramelos.
  - La cantidad de alumnos presentes.
- El programa debe calcular e imprimir:
  - Cuántos caramelos completos le corresponden a cada alumno (sin romper caramelos).
  - Cuántos caramelos sobran en la bolsa.
- **Ejemplo de Salida Esperada:**

```
Cantidad de caramelos: 53
Cantidad de alumnos: 5
Cada alumno recibe: 10 caramelos.
Sobran en la bolsa: 3 caramelos.
```

Python: Ejercicios

Ejercicio 12: Verificador de Elegibilidad para Descuento

Objetivo: Construir expresiones booleanas complejas usando `and`, `or`, `not` y operadores de comparación.

- **Enunciado:** Una tienda ofrece un **descuento especial** a los clientes si cumplen **cualquiera** de las siguientes condiciones:
  - Es cliente mayor de 65 años.
  - Es estudiante **Y** la compra supera los 50 euros.
- Escribe un programa que solicite al usuario:
  - Su edad (`int`).
  - Si es estudiante (pedir `"si"` o `"no"` y convertirlo a un valor booleano).
  - El monto total de su compra (`float`).
- El programa debe evaluar la condición e imprimir un mensaje booleano (`True` o `False`) que indique si tiene derecho al descuento.
- **Ejemplo de Salida Esperada:**

```
Edad: 20
¿Es estudiante? (si/no): si
Monto de compra: 65.5
¿Aplica descuento?: True
```

Python: Ejercicios

Test ejercicio 12: Creamos un documento con una tabla para registrar las diferentes opciones y así probar el código

edad	estudiante	compra	**descuento**
70	no	60	True
…			

Python: Ejercicios

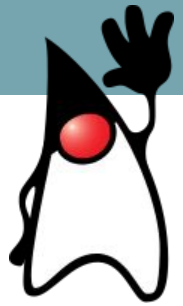
Ejercicio 13: Generador de Tique de Compra con Impuestos y Propina

Objetivo: Combinar todos los conceptos vistos en un solo script estructurado.

- **Enunciado:**
  Crea un programa para una cafetería que genere el desglose final de una cuenta.
  Debe pedir:
  - Nombre del cliente (str).
  - Nombre del producto (str).
  - Precio unitario (float).
  - Cantidad comprada (int).
  - Porcentaje de IVA aplicable (ej. 21 para 21%).
  - ¿Desea incluir propina opcional de 2€? (pedir "si" o "no").
- El programa debe calcular:
  - Subtotal básico (Precio x Cantidad).
  - Monto del IVA (Subtotal x IVA/100).
  - Total final con IVA y propina (si aplica).
  - Una validación lógica que devuelva True si el total gastado supera los 30€ (considerado "Cliente VIP").
- **Ejemplo de Salida Esperada:**

```
==================================
        TIQUE DE CAFETERÍA
==================================
Cliente: María
Producto: Café Latte x 4
----------------------------------
Subtotal: 12.0 €
IVA (21%): 2.52 €
Propina: 2.0 €
TOTAL A PAGAR: 16.52 €
----------------------------------
¿Supera el umbral VIP (>30€)?: False
==================================
```

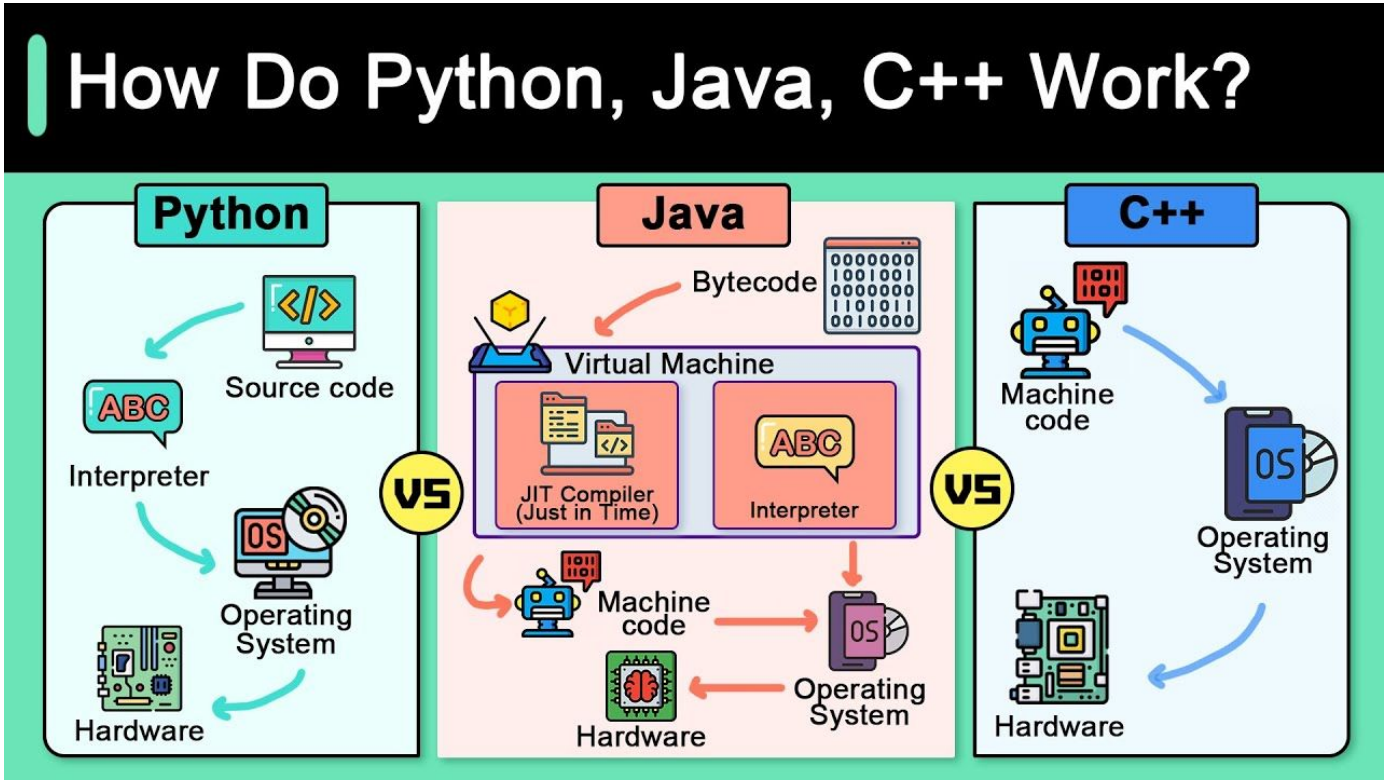
Java: ¿Por qué Java?



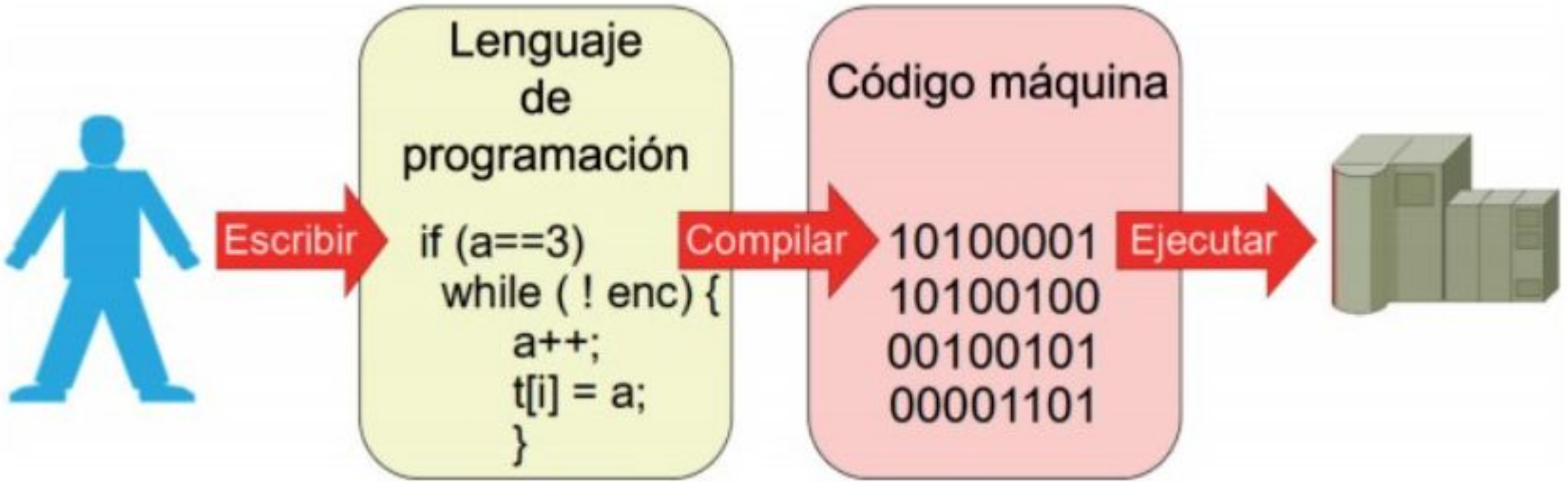
[Índice TIBOE]

	Java	**C++**	**Python**
POO	Pura	Soporta	Soporta
Propósito	General: web, móvil, backend, empresa	Específico: videojuegos, rendimiento	Específico: web, IA, Machine learning
Aprendizaje	Intermedio	Difícil	Fácil
Multiplataforma	Sí	No	Interpretado
Popularidad	Alta	Alta	Alta

Máquina Virtual de Java

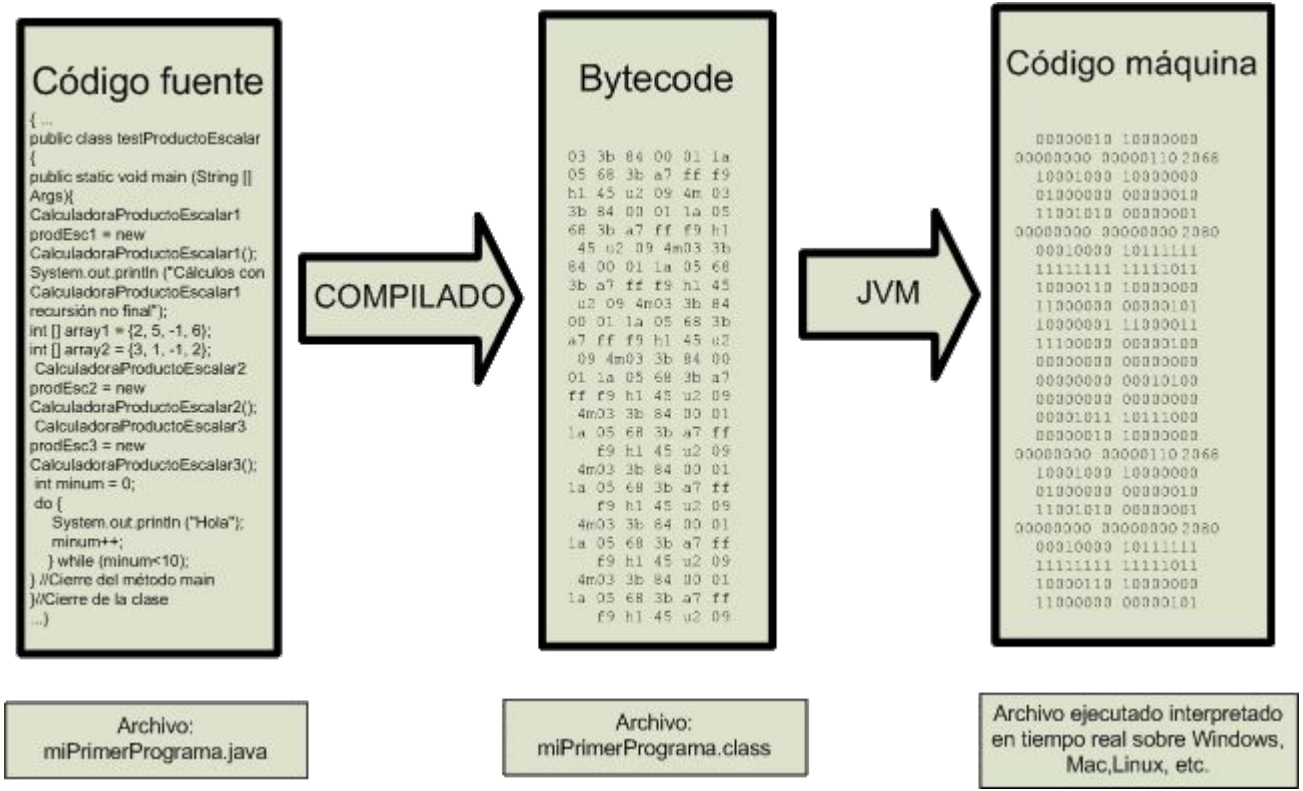


Código fuente vs código máquina



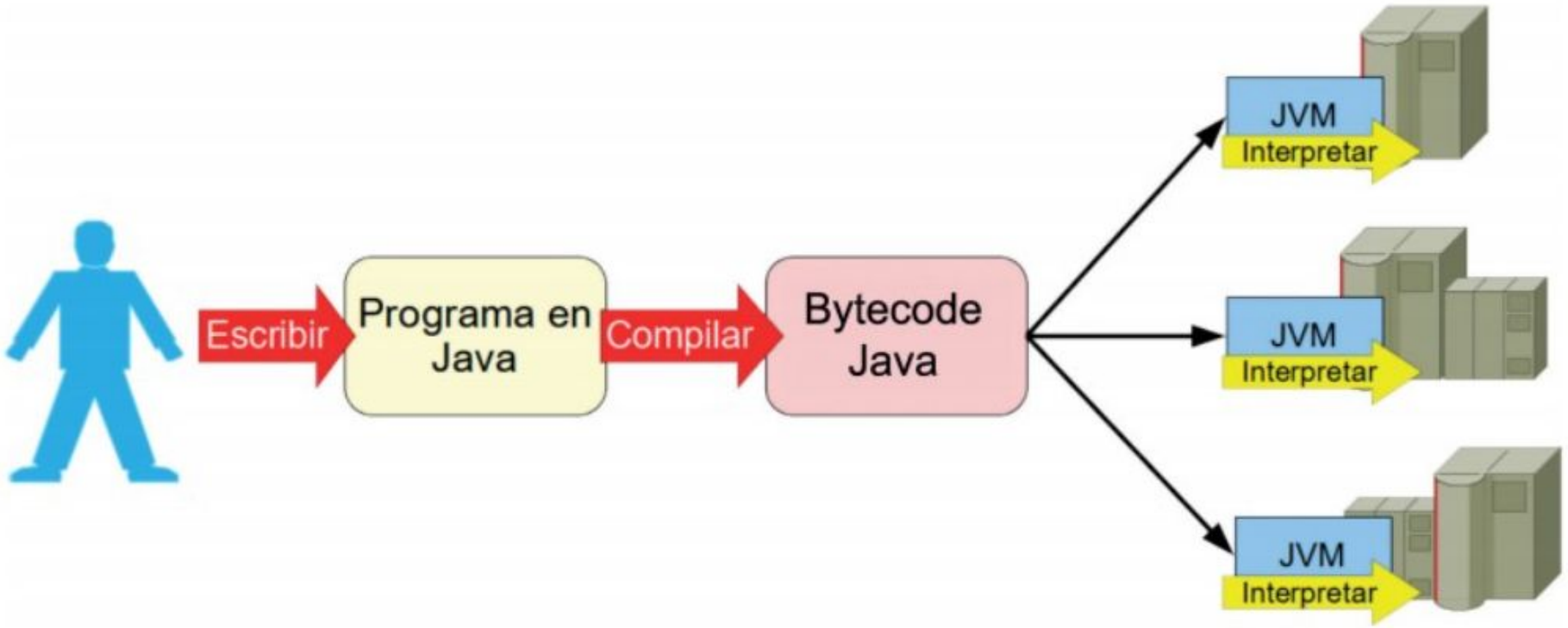
Cada plataforma (HW+SW) entiende sólo su conjunto de instrucciones

Código fuente vs código máquina

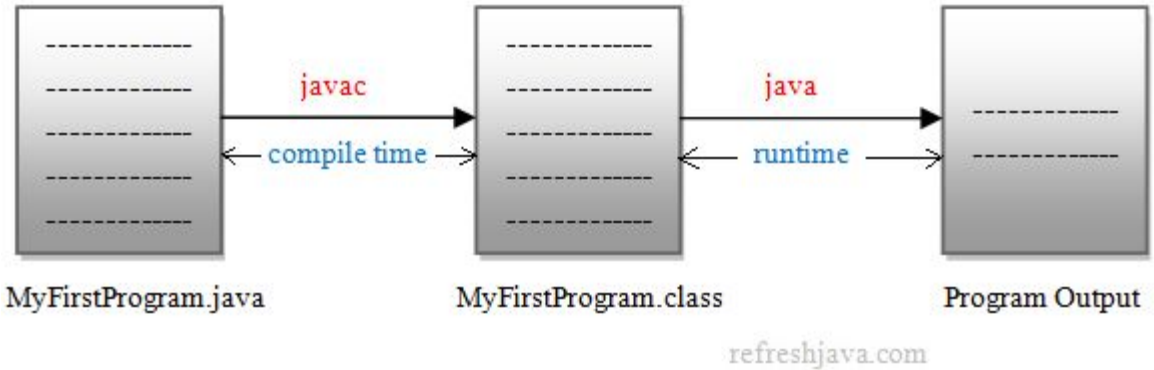




Multiplataforma



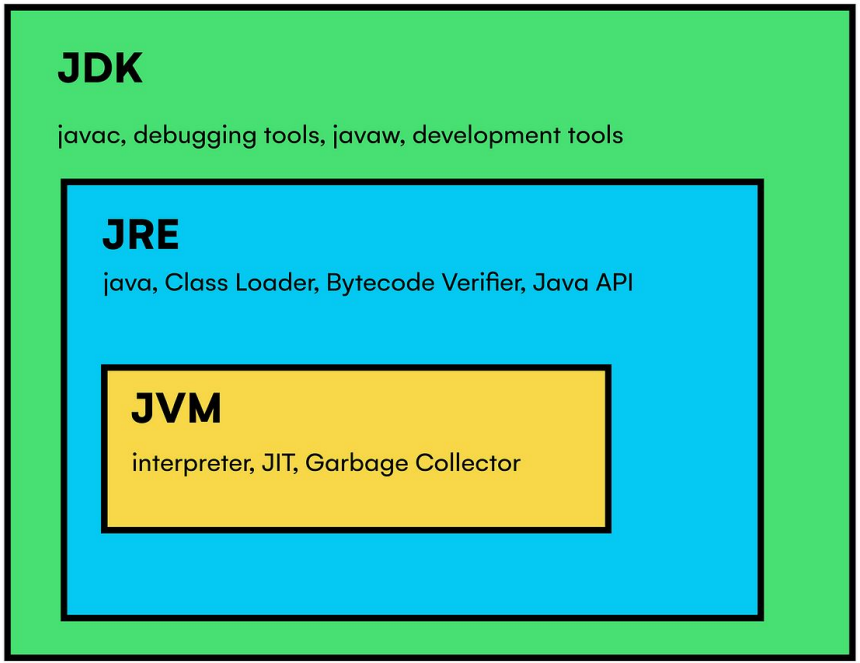
Tiempo de compilación vs tiempo de ejecución

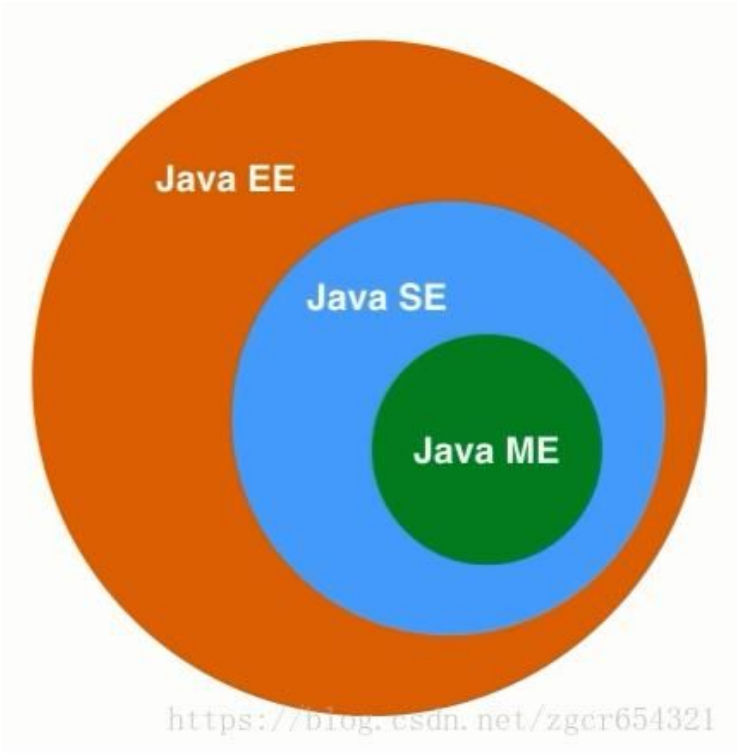


Tiempo de compilación vs tiempo de ejecución

	Compile time	**runtime**
Definición	Código fuente se traduce a código máquina	Instrucciones compiladas o interpretadas se ejecutan
Tareas	- Comprobación de sintaxis - Optimización del código - Generación de código binario	- Ejecución de las instrucciones - Asignación y liberación de memoria - Interacción con el usuario o sistema operativo
Errores	Sintaxis, declaraciones de variables incorrectas, problemas de tipos de datos, etc.	Excepciones (división por cero, acceso a memoria no válida, etc.)

Instalación de Java





Java: IDE



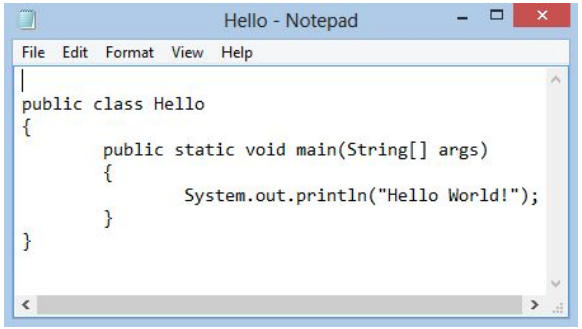
Apache NetBeans

Fits the Pieces Together

Development Environment, Tooling Platform and Application Framework.

Consola

1. Escribir el código fuente → Fichero de texto con extensión .java



2. Compilar código fuente (.java) → Instalar compilador (JDK)

```
>javac HelloWorld.java
```

3. Ejecutar código ejecutable o bytecode (.class) → Instalar JVM (JRE)

```
>java HelloWorld
```

Java: La Estructura Mínima

La Estructura Mínima:

```java
public class NombrePrograma {
    public static void main(String[] args) {
        // Aquí dentro va todo el código que se ejecuta
    }
}
```

- *Regla:* El nombre de la clase (`NombrePrograma`) debe coincidir **exactamente** con el nombre del archivo (`NombrePrograma.java`). Java distingue entre mayúsculas y minúsculas (*case-sensitive*).

Java: Sintaxis

Concepto	Java	Python
Programa	`class Main { ... }`	No necesita clase
Punto de entrada	`public static void main(String[] args)`	El código puede ejecutarse directamente
Bloques	`{ }`	Indentación
Fin de instrucción	`;`	No se necesita
Comentarios	`// comentario`	`# comentario`

Palabras reservadas

abstract	continue	float	native	strictfp	void
assert	default	for	new	super	volatile
boolean	do	if	package	switch	while
break	double	implements	private	synchronized	yield
byte	else	import	protected	this	
case	enum	instanceof	public	throw	
catch	extends	int	return	throws	
char	final	interface	short	transient	
class	finally	long	static	try	

literales reservados: true, false, null

deprecadas: const, goto

Tipos primitivos

Nombre	Tipo	Tamaño	Defecto	Ejemplo	Rango
boolean	Lógico	1 bit	false	*boolean salir = true;*	true - false
char	Carácter	2 bytes	null	char c = ‘a’;	Unicode
byte	Entero	1 byte	0	byte edad = 30;	[-128, 127]
short	Entero	2 bytes	0	short distancia = 1500;	[-32.768, 32.767]
int	Entero	4 bytes	0	int poblacionMundial = 8_000_000_000;	$[-2^{63}, 2^{63}-1]$
long	Entero	8 bytes	0	long cuentaBancaria = 987654321012345678L;	$[-2^{63}$ a $2^{63}-1]$
float	Real	4 bytes	0	float piAproximado = 3.14159f;	6 a 7 dígitos decimales
double	Real	8 bytes	0	double velocidadDeLuz = 299792.458;	15 a 16 dígitos decimales

Identificador

Nombran variables, funciones, clases y objetos; cualquier cosa que el programador necesite identificar o usar.

En Java, un identificador comienza con una letra, un subrayado (_) o un símbolo de dólar ($). Los siguientes caracteres pueden ser letras o dígitos.

Java es sensible a mayúsculas

Java Code Conventions

- Nombre variables y métodos: lowerCamelCase
- Nombre clases: UpperCamelCase

Declaración de variables

Para guardar información creamos variables.

Java es fuertemente tipado, siempre hay que indicar el tipo:

```
tipo nombreDeVariable;
```

double precio; ⇒ Declaración

precio = 2.50; ⇒ Inicialización

double precio = 2.50; ⇒ Todo en una línea

¡Hay que inicializar una variables antes de usarla!

Tipos de datos: variables y memoria

00	00111000
01	11111101
02	01011110
03	10000111
…	

Paso de decimal a binario:

00111000 11111101 01011110 10000111

Dar un tipo de dato a una variable es la forma de decir cuánto va a ocupar esa variable en memoria.

Ejemplo:

int telefono = 956128903;

El tipo entero ocupa 4 bytes

Java: Variables

Ejemplo de cómo cambiamos el valor de una variable

```java
public class ControlVida {
    public static void main(String[] args) {
        int vidaJugador = 100; // Declaración e inicialización
        System.out.println("Vida inicial: " + vidaJugador);

        // El jugador recibe un impacto (Modificación de variable)
        vidaJugador = 75;
        System.out.println("¡Impacto recibido! Vida actual: " + vidaJugador);
    }
}
```

Ejercicio 14

En el mismo proyecto del Hola Mundo, creamos un nuevo fichero: ConversorEdadPerro.java con el siguiente código:

```java
public class ConversorEdadPerro {
    public static void main(String[] args) {
        int edadHumana = 4;
        int factorPerruno = ___; // Completar con el valor 7
        int edadPerruna = edadHumana * ___; // Completar operación

        System.out.println("Edad de la mascota en años perrunos: " + ___);
    }
}
```

Rellenar el código fuente incompleto para calcular la edad perruna de una mascota.

Ejercicio 15

Crear un programa GestionInventario.java que declare la cantidad de pociones de un jugador (`int`), su precio unitario (`double`) y un estado de mochila llena (`boolean`).

Simular la compra de 3 pociones de la siguiente forma:

- Descontando el importe de un total de oro guardado en otra variable
- Mostrando el estado final de la mochila como llena.

Rango

```
byte a = 127;
a = a + 1;
```

¿Cuánto vale a?

byte → [-128 - 127] → a = -128

Constantes

final tipo nombreDeVariable;

final double PI = 3.141592;

No se puede cambiar el valor una vez inicializada

Comentarios

Comentario multilínea: cualquier texto incluido entre los caracteres `/*` (apertura de comentario) y `*/` (cierre de comentario) será interpretado como un comentario, y puede extenderse a través de varias líneas.

Comentario hasta final de línea: todo lo que sigue a los caracteres `//` hasta el final de la línea se considera un comentario.

```
/* esto es un comentario
que se extiende
durante varias líneas */
int numeroPaginas; //declaramos la variable numeroPaginas como un entero
/** este comentario será utilizado en caso de utilizar una herramienta
de generación automática de documentación */
```

Salida y Entrada de datos

Salida por consola:

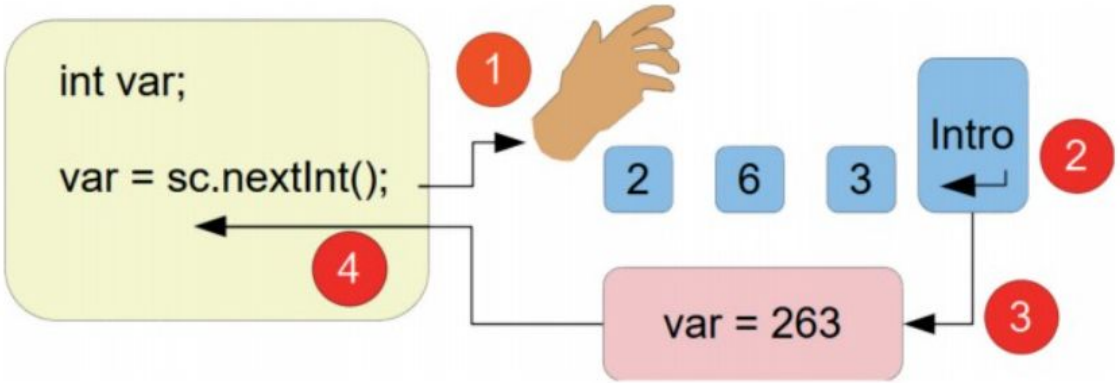
```
System.out.println("Hola."); //inserta un retorno de carro
System.out.print("La hora\ndel sistema es:\n"); //dos retornos de carro
System.out.print(queHoraEs);
```

```
Hola.
La hora
del sistema es:
10:20:32.294
```

```
java.time.LocalTime queHoraEs = java.time.LocalTime.now();
```

Salida y Entrada de datos

Clase Scanner




```
Scanner sc = new Scanner(System.in); //crea el nuevo escáner
double numero; //declaramos la variable numero
numero = sc.nextDouble(); //se detiene la ejecución del programa hasta que
//escribimos un número en el área de Output y pulsamos Intro.
//ahora disponemos del valor introducido, a través de la variable numero
System.out.println("Ha escrito: " + numero);
```

Ejercicio 16

Diseñar un programa que pida un número al usuario por teclado y a continuación lo muestre por consola.

```
Introduzca un número:
5
El número es: 5
BUILD SUCCESSFUL (total time: 5 seconds)
```

Operadores aritméticos

Símbolo	Descripción
+	Suma
+	Más unario: positivo
–	Resta
–	Menor unario: negativo
*	Multiplicación
/	División
%	Módulo
++	Incremento en 1
––	Decremento en 1

```
a ++;
b --;
```

es equivalente a:

```
a = a + 1;
b = b - 1;
```

Ejercicio 17

Nivel 1 (`CalculoDescuento.java`): Calcular el precio final de una armadura de juego de 120 créditos aplicando una constante `DESCUENTO = 0.15` (15%).

Nivel 2 (`SimuladorCrono.java`): Dado un total de segundos (ej. `3725` segundos), utilizar operadores aritméticos y de módulo (`/` y `%`) para descomponerlo y mostrar en pantalla cuántas horas, minutos y segundos exactos representa.

Ejercicio 18

Escribir una aplicación que pida el año actual y el de nacimiento del usuario. Debe calcular su edad, suponiendo que en el año en curso el usuario ya ha cumplido años.

Ejercicio 19

El tipo short permite almacenar valores comprendidos entre -32768 y 32767. Escribir un programa que compruebe que el rango de valores de un tipo se comporta de forma cíclica, es decir, el valor siguiente al máximo es el valor mínimo.

Ejercicio 20

Crear una aplicación que calcule la media aritmética de dos notas enteras. Hay que tener en cuenta que la media puede contener decimales.

Ejercicio 21

Diseñar una aplicación que calcule la longitud y el área de una circunferencia. Para ello, el usuario debe introducir el radio (que puede contener decimales).

Recordamos:

$$\text{Longitud} = 2\ \pi\ \text{radio}$$
$$\text{Área} = \pi\ \text{radio}^2$$

Declara PI como una constante.

Nivel 2: Alternativamente usa la clase Math de Java.

Operadores relacionales

Símbolo	Descripción
==	Igual que
!=	Distinto que
<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que

Sólo devuelven true o false

Ejercicio 22

Realizar una aplicación que solicite al usuario su edad y le indique si es mayor de edad (mediante un literal booleano: true o false).

Operadores lógicos

Símbolo	Descripción
&&	Operador and: Y
||	Operador or: O
!	Operador not: negación

También devuelven true o false, pero además sus operandos también son booleanos.

Operadores lógicos

a	b	a && b
falso	falso	falso
cierto	falso	falso
falso	cierto	falso
cierto	cierto	cierto

a	b	a || b
falso	falso	falso
cierto	falso	cierto
falso	cierto	cierto
cierto	cierto	cierto

a	!a
falso	cierto
cierto	falso

Ejercicio 23

¿Cuánto vale cada una de estas expresiones?

```
3 <= 5 && 2 == 2      → true
3 <= 5 && 2 >10       → false
1 != 2 || 5 < 3       → true
!(1 < 2)              → false
```

true || (cualquier condición) → true
false && (cualquier condición) → false

Evaluación perezosa

Ejercicio 24

Diseñar un algoritmo que nos indique si podemos salir a la calle.

Existen aspectos que influyen en esta decisión: si está lloviendo y si hemos terminado nuestras tareas.

Solo podremos salir a la calle si no está lloviendo y hemos finalizado nuestras tareas. Existe una opción en la que, indistintamente de lo anterior, podremos salir a la calle: el hecho de que tengamos que ir a la biblioteca (para realizar algún trabajo, entregar un libro, etc.).

Solicitar al usuario (mediante un booleano) si llueve, si ha finalizado las tareas y si necesita ir a la biblioteca. El algoritmo debe mostrar mediante un booleano (true o false) si es posible que se le otorgue permiso para ir a la calle.

Operadores opera y asigna

Símbolo	Descripción
+=	Suma y asigna
-=	Resta y asigna
*=	Multiplica y asigna
/=	Divide y asigna
%=	Módulo y asigna

```
x *= 2;
```

es equivalente a:

```
x = x * 2;
```

Ejercicio 25

Un frutero necesita calcular los beneficios anuales que obtiene de la venta de manzanas y peras. Por este motivo, es necesario diseñar una aplicación que solicite las ventas (en kilos) de cada semestre para cada fruta. La aplicación mostrará el importe total sabiendo que el precio del kilo de manzanas está fijado en 2,35 € y el kilo de peras en 1,95 €.

Operadores preincremento y postincremento

```
int a, b, c;
a = 1;
b = a++;
c = ++a;
```

¿Cuánto valen a, b y c?

```
int a, b, c; //declaramos las variables de tipo entero
a = 1; //a la variable "a" le asignamos 1
b = a ++; //primero asignamos el valor de "a" a "b", y después incrementamos "a"
c = ++ a; //primero incrementamos "a", y después asignamos su valor a "c"
```

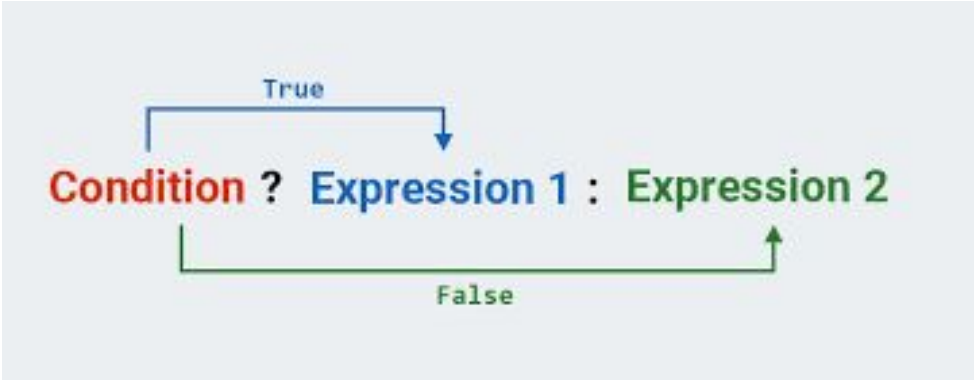
$a \rightarrow 3, b \rightarrow 1, c \rightarrow 3$

Precisión de los tipos de datos


```
double a = 1.0/10.0 + 2.0/10.0; //el resultado debería ser 3/10
a = a * 10.0; //el resultado debería ser 3
System.out.println(a); //muestra 3.000000000000004
```

Errores de precisión

Operador ternario




```
int a, b;
a = 3 < 5 ? 1 : -1; // 3 < 5 es cierto: a toma el valor 1
b = a == 7 ? 10 : 20; // a (que vale 1) == 7 es falso: b toma el valor 20
```

Ejercicio 26: Calculadora de precio de entrada

Crea un programa en Java que calcule el **precio final de una entrada de cine** en función de la edad del cliente.

El programa debe solicitar por teclado:

- El nombre del cliente.
- Su edad.

Aplica las siguientes tarifas:

Edad	Precio
Menor de 12 años	5 €
Entrada general	8 €

El programa debe mostrar:

Cliente: Ana
Edad: 70
Precio de la entrada: 8€ o 6 € según la tabla aplicada.

Edad	Precio
Menor de 12 años	5 €
Entre 12 y 64 años	8 €
65 años o más	6 €

Para calcular el precio debes utilizar **el operador ternario `?:`**

```
double precio = condicion ? valorSiVerdadero : valorSiFalso;
```

Precedencia de operadores

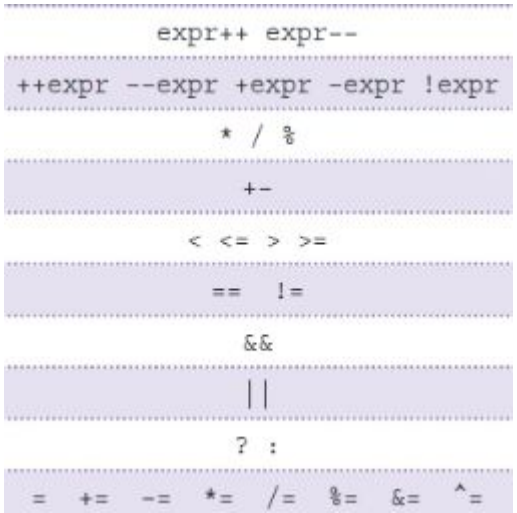
Descripción	Operador
Posfijos	`expr++ expr--`
Unarios prefijos	`++expr --expr +expr -expr !expr`
Aritméticos	`* / %`
Aritméticos	`+ -`
Relacionales	`< <= > >=`
Comparación	`== !=`
AND lógico	`&&`
OR lógico	`||`
Ternario	`? :`
Asignación	`= += -= *= /= %= &= ^=`

Ejercicio 27

1. Escribe un programa en Java que evalúe las siguientes expresiones.
2. Muestra el resultado de cada expresión y explica qué operadores tienen mayor precedencia.
3. Usa operadores aritméticos, relacionales, lógicos, y operadores de asignación.

Expresiones:

1. `10 + 5 * 2 > 20 && 4 == 4`
2. `!(7 + 3 > 10) || 3 * 2 <= 6`
3. `10 / 2 + 3 * 5 == 19 && true`
4. `int x = 5; x += 3 * 2;`
5. `boolean b = false; b = !b || 7 % 2 == 1;`



Conversión de tipos

Toda variable tiene un tipo y sólo admite valores de ese tipo

```
int a = 1;
double x = 2.3;
```

Sin embargo, podemos realizar conversiones entre tipos de dos formas:

- Implícita: de un tipo de menor capacidad a uno de mayor
- Explícita: de un tipo de mayor capacidad a uno de menor ⇒ hay pérdida de información

`int a = 2.3` ⇒ Error de compilación

Asigna un real a una variable entera

Conversión de tipos

Conversión implícita

```
int a = 1;
double x = a;
```

La conversión de tipos también se conoce como 'Casting'

Conversión explícita

```
double x = 2.3;
int a = (int)x;
```

⇒ hay pérdida de información pero no da error

Conversión de tipos

¿Qué tipo de conversión está ocurriendo aquí?

```
int a = 'a';
```

a	97
…	

UNICODE → 'a'

¿que pasa si hacemos al revés?
(asignar a un char un int)

Ejercicio 28

Escribir un programa que solicite las notas del primer, segundo y tercer trimestre (notas enteras que se solicitarán al usuario).

El programa debe mostrar la nota media del curso como aparece en el boletín de calificaciones (solo la parte entera) y como se usa en el expediente académico (con decimales).

Mini-Proyecto integrador

Desarrollar un programa pequeño que combine varios de los conceptos aprendidos: pedir datos al usuario, almacenarlos en variables, realizar cálculos con operadores, usar alguna constante, mostrar resultados, añadir comentarios, realizar conversión de tipos. (Ej: Una calculadora básica, un conversor simple de unidades).

Requisitos:

- Se debe realizar en parejas en una sesión de clase
- Un sólo repositorio en GitHub compartido por la pareja
- Usar un lenguaje de programación (Python, Java, Javascript…)

Evaluación:

- Se expondrá en clase mediante una demo de no más de 5 minutos
- Habrá que responder a las preguntas de la audiencia
- La calificación es la misma para ambos miembros de la pareja
- Ambos miembros deben responder por separado con lo que ambos tienen que conocer todo el proyecto

Examen

Ejercicio de Java a resolver con los conceptos estudiados en clase.

IDE: Netbeans

Entrega: Moodle o GitHub

Recursos y referencias

- Libro que tenéis en la biblioteca y en el departamento
- Diapositivas en Moodle
- Hoja de Recursos
- Ejercicios de refuerzo y de ampliación
- Vuestro portfolio

Fin